AGV2 Kayaba — Compiled Release Guide

How the locked release made by `build_release.sh` works for the customer, and how the developer builds, updates and rolls it back. The AGV only ever runs a compiled program (`/opt/agv2/main.bin`); the Python source stays with the developer. 9 Oct 2026.

1. Customer side (after hand-over)

What the customer gets

- **Power on and it runs.** The controller starts automatically at boot as the `agv-controller` service and restarts itself within 3 s if it ever crashes.
- **The same web dashboard** at `http://<agv-ip>:5000`: HOME, MANUAL, IO, PARAMS and ERRORS work exactly as before, including the restart button.
- **The same physical controls:** RESET, branch buttons (start AUTO), CALL START, the pusher button DI0, E-stop, mode selector.
- **Operator functions without a password:** HOME, MANUAL (jog, bypass buttons, pusher, AUTO REVERSE), IO lamps, ERRORS, and the physical pusher button DI0.

What the customer cannot do

- **See the code.** `/opt/agv2` holds only `main.bin` and libraries — no .py files — and a normal Ubuntu user cannot open the folder.
- **Change the profile** (RFID tags, speeds, IO) without the developer's admin (sudo) password.
- **Log in as the service account** agv — it has no login shell.
- **Use PARAMS, IO TEST or calibration** without the engineer password (ENGINEER button in the web bottom bar, 30-minute session).

> **Note**
> • If the customer needs to log into Ubuntu, create a normal user **without sudo** and never share the admin password. If they don't need to log in, create no account at all.

2. Developer side

First release on the AGV

```
# on the AGV's Ubuntu PC
git clone https://github.com/joelevander10/AGV-Kayaba-2-Sensor-.git ~/agv2
cd ~/agv2
./build_release.sh
```

The script installs the build tools and Nuitka, compiles everything into `main.bin`, installs it to `/opt/agv2` (owned by root, readable only by the agv service account), creates the agv user, writes the `agv-controller.service` systemd unit and the restart-button sudoers rule, starts the service and checks the dashboard answers. `./build_release.sh --build-only` only compiles.

Test everything on the real AGV — AUTO, reverse + CC hitch, pusher, lidar, all web pages — then remove the source:

```
rm -rf ~/agv2                          # source + .git (your copy stays on GitHub)
gh auth logout; rm -f ~/.git-credentials
sudo systemctl disable --now ssh       # unless you want remote access
```

Updating later

- Change the code on the Mac, commit and push in GitHub Desktop as usual.
- On the AGV: clone again, build, test, delete the source:

```
gh auth login
git clone https://github.com/joelevander10/AGV-Kayaba-2-Sensor-.git ~/agv2
cd ~/agv2 && ./build_release.sh
rm -rf ~/agv2 && gh auth logout
```

Everyday commands on the AGV

Task	Command
Is it running?	`sudo systemctl status agv-controller`
Live log	`sudo journalctl -u agv-controller -f`
Restart	`sudo systemctl restart agv-controller`
Stop	`sudo systemctl stop agv-controller`
Change a tag or speed on site	`sudo nano /opt/agv2/profiles/agv2_kim.json (then restart)`
Only compile, don't install	`./build_release.sh --build-only`

Engineer password (web PARAMS / IO TEST / calibration)

Asked for automatically on the first `build_release.sh` install and kept across rebuilds. Set or change it any time (no restart needed); only a hash is stored, in `/opt/agv2/engineer_auth.json`:

```
sudo AGV_ID=agv2_kim /opt/agv2/main.bin --set-engineer-password
```

Rolling back a bad build

Every run keeps the previous release as `/opt/agv2.old-<date-time>`:

```
sudo systemctl stop agv-controller
sudo mv /opt/agv2 /opt/agv2.bad
sudo mv /opt/agv2.old-<date-time> /opt/agv2
sudo systemctl start agv-controller
```

3. Watch out for

Note
• **Profile edits made on the AGV are replaced by the next build** — the script installs the profile from the repo. Copy on-site changes back into `agv2_kim.json` in the repo before rebuilding (the old release folder still has them if you forget).
• **Always build on the AGV itself**, or on a PC with the same Ubuntu version and CPU. A binary built elsewhere may not run.
• **Keep the source safe:** private GitHub repo plus a backup on the Mac. Nothing on the AGV can recover the code.
• **Stronger protection:** BIOS/UEFI password, USB boot disabled, and LUKS disk encryption with TPM unlock so the SSD can't be read in another computer.
• **Legal protection:** state in the sale or licence agreement that the software stays your property, licensed for this AGV only, with no copying, decompiling or reverse engineering.